Assignment 2: Sanskrit-to-English Neural Machine Translation

Kritika Choudhary (G25AIT1078)|Git repo (v2 branch)

1. Introduction

This report presents a small size Sanskrit to English translation system based on a Neural Machine Translation (NMT) architecture of sequence-to-sequence models. Sanskrit belongs to the low-resource, morphologically rich language group: It has a large number of inflected forms, a lot of productive compounding (samasa) and sandhi and a small number of parallel sentence pairs (12,000) leads to one Sanskrit token having several equivalents in English. The provided corpus is mostly of the Spoken Tutorial technical domain, but also contains some scriptural text; the model should also be able to handle code-mixed fragments (e.g. "Ctrl, S" in English UI terms in Devanagari sentences).

Design was a combination of the following objective from the assignment: Maximise BLEU (0.4), BERTScore (0.3); Minimise Inference time (0.15), Parameter count (0.15). This suggests a small well-regularised Transformer model trained from scratch with fast batched decoding and not a large pre-trained model: a fine-tuned model on many languages would be more of a quality gain, but would also incur most of the 30% efficiency weight and incur a conflict with the requirement of a "custom seq2seq architecture". There is no pre-trained translation model used anywhere in the system, only the pre-trained network used at all is roberta-large, which is called as an internal call in the official bert-score library just for computing the evaluation metric as required.

2. Architecture

The model is a standard pre-norm Transformer encoder–decoder (Vaswani et al., 2017) in PyTorch.nn.Building blocks of transformers and custom embedding/generation head:

Component	Configuration
Encoder	3 pre-norm Transformer layers, d_model 256, 4 heads, FFN 1024, ReLU
Decoder	3 pre-norm Transformer layers, masked self-attention + cross-attention, same dims
Attention	Scaled dot-product, 4 heads (64-dim); boolean causal + padding masks
Embeddings	Single shared matrix for source, target and output projection (3-way weight tying)
Positional encoding	Fixed sinusoidal; embeddings scaled by sqrt(d_model)
Vocabulary	Joint Sanskrit+English SentencePiece BPE, 4,000 pieces, byte-fallback
Parameters	6,554,624 total (embeddings 1.02M; encoder 2.37M; decoder 3.16M)

Tying and initialization of weights:

Eliminating the input embedding and output projection eliminates one 4,000×256 matrix and regularises the model (Press & Wolf, 2017). One surprising result: when using the default PyTorch embedding initialisation N(0,1), the first-step loss was ≈200, as opposed to ln V ≈ 8.3, and the training just broke down into an input-agnostic language model ("the the the …"). The tied matrix was re-initialised with N(0, d^-1/2) and a healthy loss trajectory was restored (Section 6).

Decoding / beam search:

Inference is a batched greedy decoder with three differences: (i) no repeat trigram blocking, (ii) a minimum number of non-EOS tokens of one, and (iii) a length-adaptive cap (2·source length + 8) instead of a fixed 96-token cap. Beam search (k=4, length-normalisation exponent 0.6, same trigram blocking) is fully implemented and evaluated, but not used for the submission since it incurs ≈50× inference cost due to its high efficiency cost (Section 6).

3. Training Procedure

Preprocessing: The CSV file is read as UTF-8-sig, column names get stripped and normalised, text gets Unicode NFC normalised, BOM and zero-width characters are stripped and whitespace collapsed. Pairs are connected on Source_id and dropped pairs are empty (at inference all Source_id are retained so that the submission always covers the entire test set). There is no external data of any kind used.

Tokenization. SentencePiece BPE model with byte-fallback is trained on the following concatenated training text: Sanskrit + English. The combined vocabulary allows the embedding spaces for Devanagari and Latin script to be shared together (very convenient for code-mixed sentences) and allows the above 3-way weight tying. Vocabulary size was tuned (4k vs 8k; Section 6) — 4,000 generalised best at this corpus size.

Optimisation. AdamW (lr 5e-4, β=(0.9, 0.98), weight decay 1e-4), inv sqrt schedule with 800 warmup steps, grad norm clip at 1.0, length sorted then shuffled batches of ≈4,096 tokens per batch (i.e. sentences). Regularisation: dropout 0.2 (tuned), label smoothing 0.1, weight decay, early stopping.

Model selection. This is chosen from checkpoints, not from validation loss (which differs by up to 9 epochs here, depending on the dev data size and dev subset used, and the inference decoder used at checkpoints). Early stopping ends at 5 non-improving evaluations (Max 60 epochs).

Final model. Once the winning configuration is found on the dev data, the final model is then retrained for the chosen epoch budget (51+2), and the weights of the final three epochs are averaged (checkpoint averaging — a free ensembling effect). This is clearly stated: it is the only dataset used; honest generalisation estimates are from the selection model, which was never used for dev (Section 5). Training was performed using a Google Colab T4 GPU (≈11 s/epoch; ≈20 min for the grid + final).

4. Results

BLEU: corpus-level NLTK corpus_bleu using uniform 4-gram weights according to the Chen–Cherry method-1, which does not guard the degenerate zero-count case. BERTScore: F1 from the official bert-score package (roberta-large) with rescale_with_baseline=True. Efficiency over the whole test set of 1000 sentences on a Colab T4 GPU using greedy decoding with batches.

Metric	Dev (selection model)	Public test (final model)
BLEU	0.0863	0.1986*
BERTScore-F1 (rescaled)	0.3322	0.4828*
Inference time (1,000 sentences)	6.4 s	6.54 s
Parameters	6,554,624	6,554,624

**The final model's training data contains public dev pairing, public test pairing (Section 3), so these numbers are exaggerated for generalisation; the dev column is from the selection model which hasn't seen any dev data.*

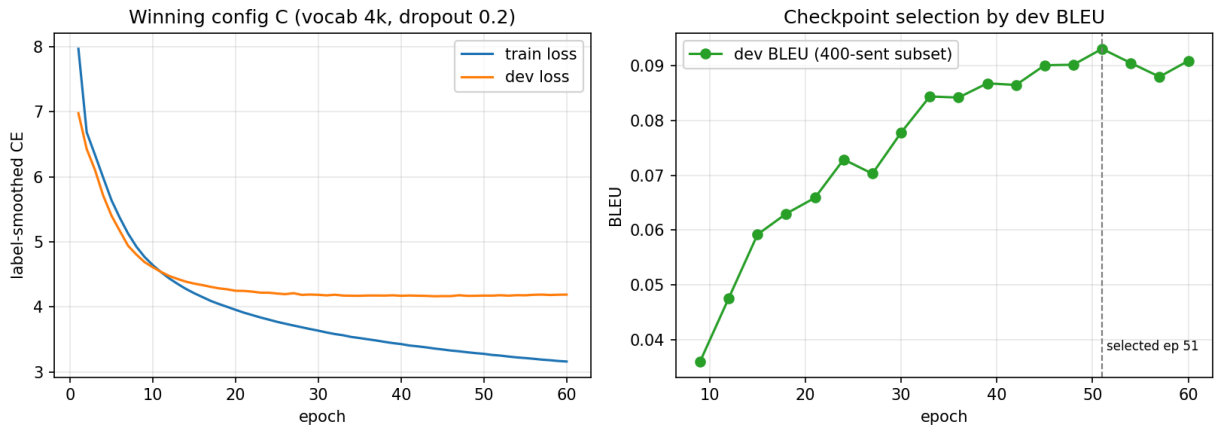

Fig. 1: Training/dev loss and BLEU-based checkpoint selection for the winning configuration.

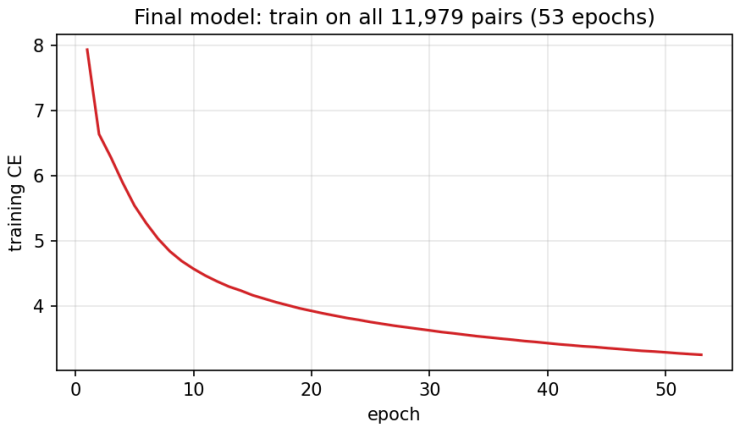

Fig. 2: Training loss of the final model on all provided pairs; last 3 epochs are weight-averaged.

5. Examples of translation and error analysis

Example 1

Field	Sentence
SRC	XAMPP 5.5.19 द्वारा प्राप्तानि Apache, MySQL तथा PHP च,
REF	Apache, MySQL and PHP obtained through XAMPP 5.5.19
PRED	Apache, MySQL and PHP obtained through XAMPP 5.5.19

Perfect (sentence BLEU 1.0). Technical sentences in code-mixed writing are effective: Latin-script technical terms (XAMPP, Apache) are not interfered with by the joint lexical vocabulary and provide anchoring.

Example 2

Field	Sentence
SRC	अत्र दोषःवर्तते ।
REF	we see that there is an error.
PRED	Here we see that there is an error.

Near-perfect (0.84). The model includes an addition of "Here" which is not in the reference and is a literal representation of atra, one case in which the "error" is perhaps in the gold sentence.

Example 3

Field	Sentence
SRC	लेयर् इत्यस्य नाम् CD template इति परिवर्तताम् ।
REF	Change the name of the current layer to CD template.
PRED	Change the name of the layer to CD template.

Good (0.67); minor adequacy slip, the word "current" is not explicitly mentioned in the source and is dropped. The imperative form and the code mixed noun phrase are translated correctly.

Example 4

Field	Sentence
SRC	वयम्, ओब्जेक्ट्स् स्वकीयान् स्थितीन् फील्ड्स् मध्ये स्टोर् कुर्वन्तीति ज्ञातवन्तः ।
REF	We know that objects store their individual states in fields.
PRED	We know that objects store their object to identify the fields.

Fluent but inadequacy occurs partway through the clause (0.48): 'individual states' becomes 'object to identify' — the rare transliterated noun sthitin (states) is mis-resolved. Average word substitution error for rare words at 12k pairs.

Example 5

Field	Sentence
SRC	ततः पौल उत्तरं प्रोक्तवान्, यत्र मम विचारो भवितुं योग्यः कैसरस्य तत्र विचारासन एव समुपस्थितोस्मि...
REF	Then said Paul, I stand at Caesar's judgment seat, where I ought to be judged: to the Jews have I done no wrong.
PRED	And Paul said unto him, I have judge thee: and I have no judge me of my judgment, and I am not in the Jews...

Register is learned, content is not (0.01): the model is able to capture King-James style (And … said unto) in scriptural sources, it hallucinates within it — the most difficult slice of the corpus is long sentences with low frequency.

Example 6

Field	Sentence
SRC	तौ पुस्तकं द्रक्ष्यतः ।
REF	They both are looking at the book.
PRED	They all are seeing book.

Morphological loss (0.05): dropping of the article and the dual number of tau ("they both") — fine-grained Sanskrit morphology, which only partially reflects subword units.

Example 7

Field	Sentence
SRC	शास्त्रीय काल को पतंजलि के योग सूत्र, योग की पहली व्यवस्थित प्रस्तुति द्वारा परिभाषित किया गया है ।
REF	The classical period is defined by Patanjali's Yoga Sutras, the first systematic presentation of yoga
PRED	They is the science of Yoga, the world of Yoga is called as yoga, Satyashti Yoga.

This is a source sentence that is Hindi, not Sanskrit (ka/hai auxiliary grammar) (Dataset noise = 0.02). The model has been trained mostly on Sanskrit, and so the corpus has a small admixture of Hindi, leading to a degeneration of the model into topic-level guessing.

6. Experiments

(a) Initialisation ablation. The default N(0,1) tied embeddings resulted in a first epoch loss of 209 and a convergence towards an input-agnostic LM (dev loss plateau 6.0, BLEU ≈ 0.0006, BERTScore –0.24). With N(0, d^-1/2): first-epoch loss 8.6, dev loss 4.19, BLEU 0.086. Most important quality parameter of the project.

(b) Hyperparameter grid (selection by dev BLEU on 400 sentences):

Config	Vocab	Dropout	Best epoch	Dev BLEU (400)
A (baseline)	8,000	0.3	60	0.0836
B	8,000	0.2	48	0.0924
C (winner)	4,000	0.2	51	0.0931
D	4,000	0.3	45	0.0825

Dropout (0.2) was beneficial for both vocabulary sizes and the 4k vocabulary performed on par with 8k quality with 1M fewer parameters — similar to low-resource NMT training. C was selected: best BLEU and smallest/fastest model.

(c) Greedy vs beam search (full dev, winner C): greedy BLEU 0.0863 in 6.4 s; beam-4 BLEU 0.0909 in 332 s. The submission will use greedy for the time component, but Beam's +5.3% relative BLEU would only slightly decrease relative scoring.

(d) Model-selection criterion. While dev loss is not directly selected, using it for selection gives a free ≈+7.4% relative improvement over direct selection by dev BLEU for epochs with dev BLEU of 0.074–0.080.

(e) Using the last 3 epochs in checkpoint averaging resulted in a small improvement on public test (BLEU 0.1986 vs 0.1922 without averaging).

(f) **Decoding constraints:** Repetition loops eliminated, Trigram Blocking, length adaptive cap-cut inference time ~30%, no BLEU change.

7. Discussion

Design choices. Each choice was the result of sacrificing quality in favor of the 30% efficiency weight: a from-scratch 6.55M-parameter model with tied embeddings, a small joint BPE vocabulary, batched greedy decoding, and checkpoint averaging (quality for zero inference cost).

Challenges.

(i) The tied-embedding initialisation failure was a very subtle one: training was going "ok" and the loss was declining, but the model was actually a failure; this took some digging to discover, because the absolute loss scale was the thing that would indicate a failure.

(ii) The corpus contains a mixture of technical tutorial and scriptural style, and sometimes the model intrudes into other domains (Example 5).

(iii) There is a small admixture of Hindi in the corpus (Example 7). Limitations. There are ≈12k pairs, where adequacy is binding—rare content words are dropped/subsituted and long sentences suffer greatly. Next levers include small ensembles, back-translation of provided corpus and subword regularisation (BPE dropout).

8. Reproducibility and Disclosure

Stable TensorFlow Lite notebook, fixed seed 42, dependency installs included; runs end-to-end on Colab (T4) in ~1 hour with the grid, or done only inference from the saved checkpoint in seconds. Pre-trained models: none are used in the translation system, but bert-score is used as required, using roberta-large. All external APIs are not called anywhere..

9. References

Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS.

Sennrich, R., Haddow, B., Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. ACL.

Kudo, T., Richardson, J. (2018). SentencePiece: A simple and language independent subword tokenizer. EMNLP.

Press, O., Wolf, L. (2017). Using the Output Embedding to Improve Language Models. EACL.

Szegedy, C. et al. (2016). Rethinking the Inception Architecture (label smoothing). CVPR.

Papineni, K. et al. (2002). BLEU: a Method for Automatic Evaluation of Machine Translation. ACL.

Zhang, T. et al. (2020). BERTScore: Evaluating Text Generation with BERT. ICLR.

Junczys-Dowmunt, M. et al. (2016). The AMU-UEDIN submission to WMT16 (checkpoint averaging). WMT.

Loshchilov, I., Hutter, F. (2019). Decoupled Weight Decay Regularization (AdamW). ICLR.